

Computación Ubicua

Introducción a Arduino

Arduino

- **Arduino** es una plataforma de prototipado electrónico de código abierto basada en hardware y software fáciles de usar. Está pensada para artistas, diseñadores, aficionados y cualquier persona interesada en crear objetos o entornos interactivos.



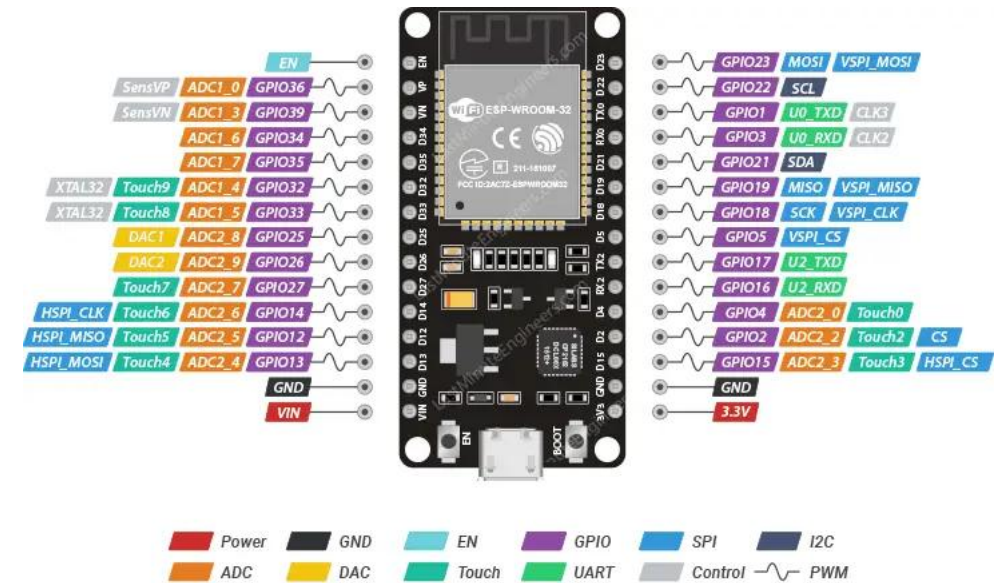
Tipos de placas más comunes

Placa	Descripción breve	Ideal para...
Arduino Uno	La más popular. 14 pines digitales, 6 analógicos.	Principiantes y proyectos básicos
Nano	Más pequeña, similar a la Uno.	Proyectos compactos
Mega	Más memoria y más pines.	Proyectos grandes (robótica, domótica)
Leonardo	Puede simular teclado/ratón.	Interfaces humanas (HID)
Due	Más potente (procesador ARM).	Proyectos más exigentes
ESP8266 / ESP32	Incluyen Wi-Fi y/o Bluetooth.	IoT y conectividad inalámbrica



Pinout

- El esquema de patillaje es de los documentos de referencia más útiles a la hora de realizar un montaje. El caso de Arduino no es una excepción, más aún teniendo en cuenta la gran cantidad de pines y modelos de placas disponibles.



ESP32 Dev. Board Pinout

Montaje

○ Existen 2 tipos de entradas:

– Digitales

- Una señal digital es una variación de voltaje entre $-V_{cc}$ a $+V_{cc}$ sin pasar por los valores intermedios
- Al valor inferior $-V_{cc}$ le asociamos un valor lógico LOW o 0 lógico
- Al valor superior $+V_{cc}$ le asociamos HIGH o 1 lógico.

– Analógicas

- Una señal analógica es una magnitud que puede tomar cualquier valor dentro de un intervalo $-V_{cc}$ y $+V_{cc}$.
- las entradas analógicas proporcionan una medición codificada en forma de un número digital (por ejemplo de 0 a 255).

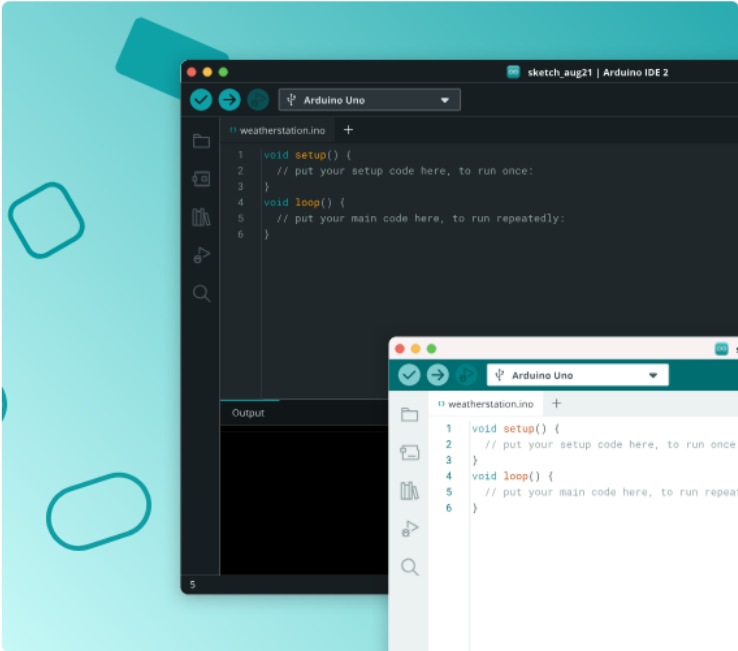
Hojas de características

- Es un documento crucial que detalla todas las especificaciones y propiedades relevantes de un sensor específico.
- Información clave que se encuentra en una hoja de características:
 - Rango de medición: Define los límites superior e inferior de los valores que el sensor puede medir con precisión.
 - Precisión: Indica la cercanía de las mediciones del sensor a los valores reales.
 - Resolución: Determina la menor variación que el sensor puede detectar.
 - Repetibilidad: Mide la consistencia de las mediciones al repetir la misma medición varias veces.
 - Linealidad: Indica si la relación entre la entrada y la salida del sensor es lineal o no.
 - Sensibilidad: Mide la respuesta del sensor a cambios en la variable medida.
 - Tiempo de respuesta: Define la velocidad con la que el sensor reacciona a cambios en la variable medida.
 - Condiciones ambientales: Especifica la temperatura, humedad y otras condiciones ambientales que el sensor puede soportar.
 - Conexiones eléctricas: Describe los pines o conexiones necesarias para alimentar el sensor y obtener sus datos.
 - Software y protocolos: Incluye información sobre el software o protocolos de comunicación necesarios para interactuar con el sensor.

Arduino IDE

Descarga del IDE

- <https://www.arduino.cc/en/software/>



**Arduino IDE 2.3.6**
[Release notes](#)

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the [Arduino IDE 2.0 documentation](#).

Windows Win 10 or newer (64-bit) 

DOWNLOAD

**Nightly Builds**
Download a preview of the incoming release with the most updated features and bugfixes. 

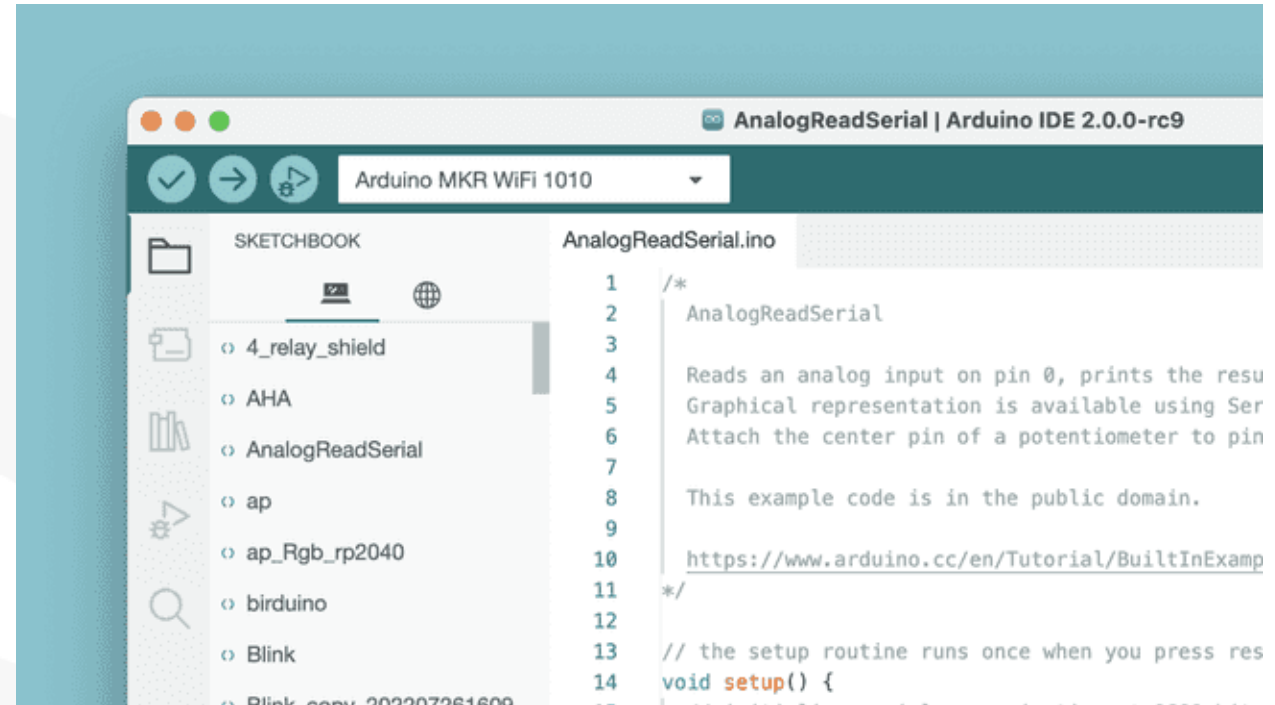
The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

Arduino IDE



Arduino IDE - Sketchbook

- Los cuadernos de Arduino se guardan como archivos .ino y deben almacenarse en una carpeta con el mismo nombre.
- El icono de la carpeta situado en la barra lateral permite acceder a los archivos del proyecto



Arduino IDE – Gestor de placas

- El gestor de placas permite buscar e instalar paquetes que permiten compilar el código y adaptarlo a las distintas placas.
- Hay varios paquetes de placas Arduino disponibles, como avr, samd, megaavr y más.



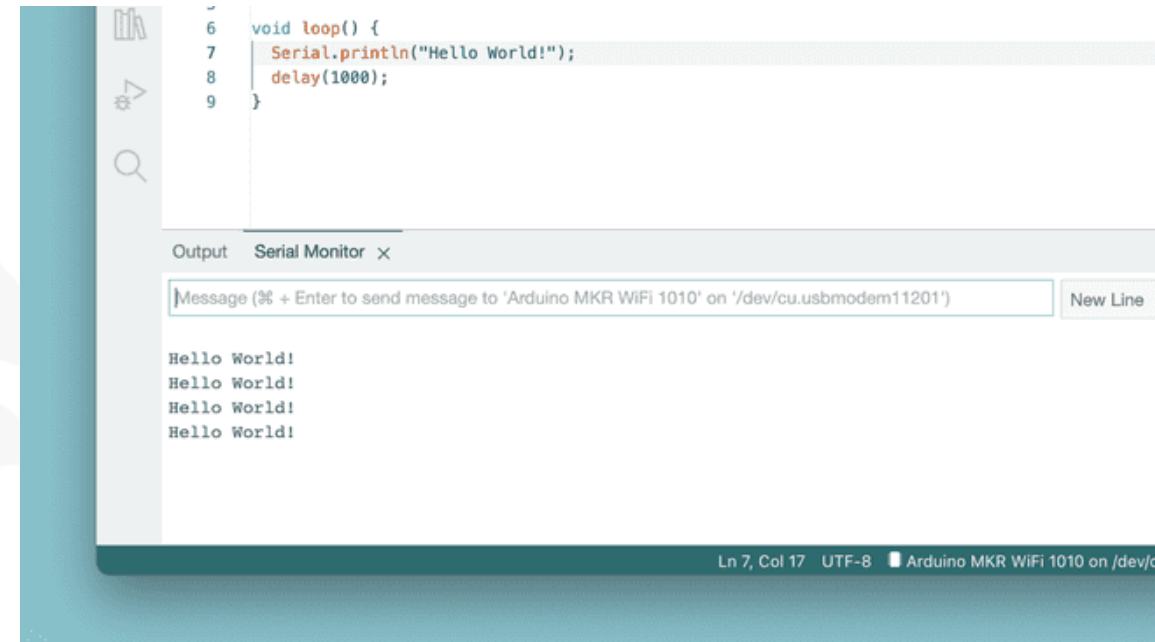
Arduino IDE – Gestor de Bibliotecas

- El gestor de bibliotecas permite explorar e instalar distintas bibliotecas que facilitan el desarrollo de programas



Arduino IDE – Monitor Serie

- El monitor serie es una herramienta que permite ver los datos que se transmiten desde la placa, por ejemplo, mediante el comando `Serial.print()`.
- Históricamente, esta herramienta se encontraba en una ventana separada, pero ahora está integrada en el editor. Esto facilita la ejecución simultánea de varias instancias en el ordenador.



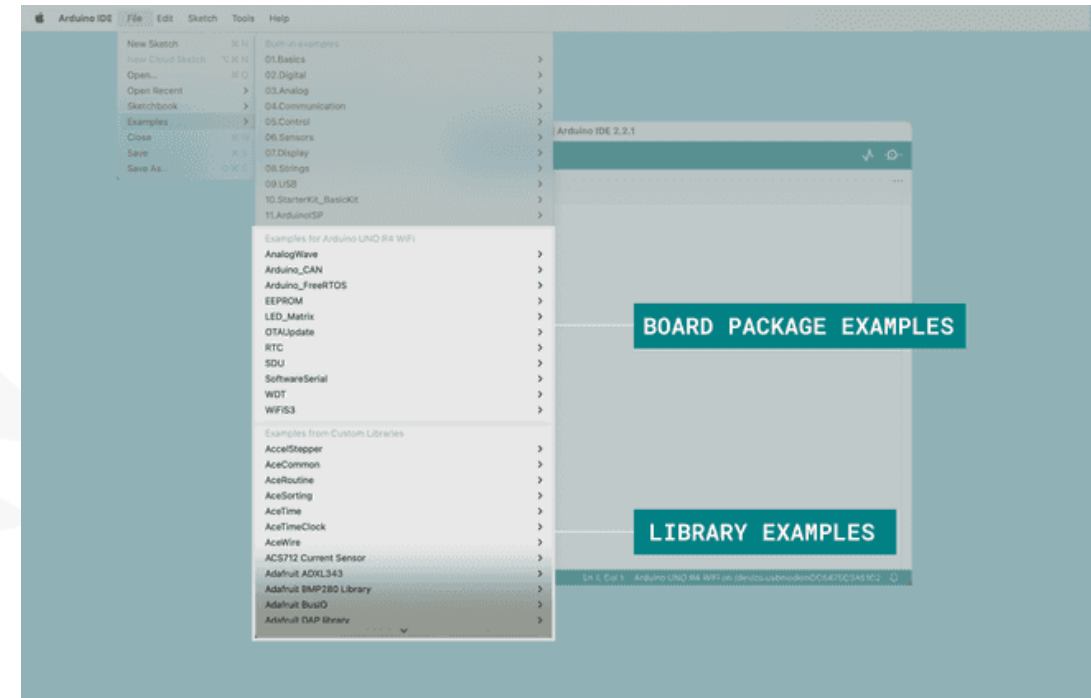
Arduino IDE – Serial Plotter

- La herramienta Serial Plotter es ideal para visualizar datos mediante gráficos y supervisar, por ejemplo, los picos de tensión.
- Puede supervisar varias variables simultáneamente, con opciones para habilitar solo determinados tipos.



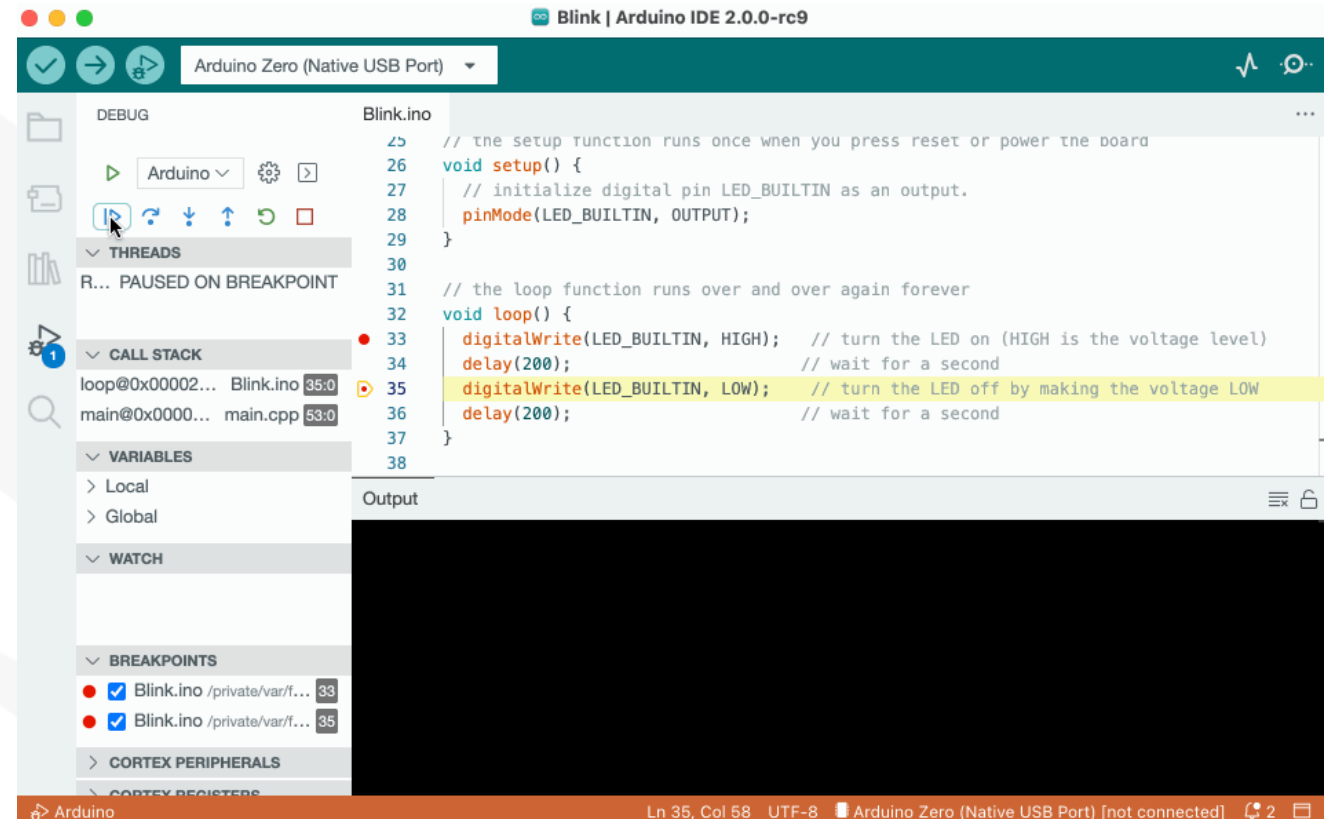
Arduino IDE – Ejemplos

- Una parte importante de la documentación de Arduino son los ejemplos de proyectos que se incluyen con las bibliotecas. Estos muestran ejemplos de las funciones utilizadas en la práctica, ilustrando el uso previsto y las características de una biblioteca.



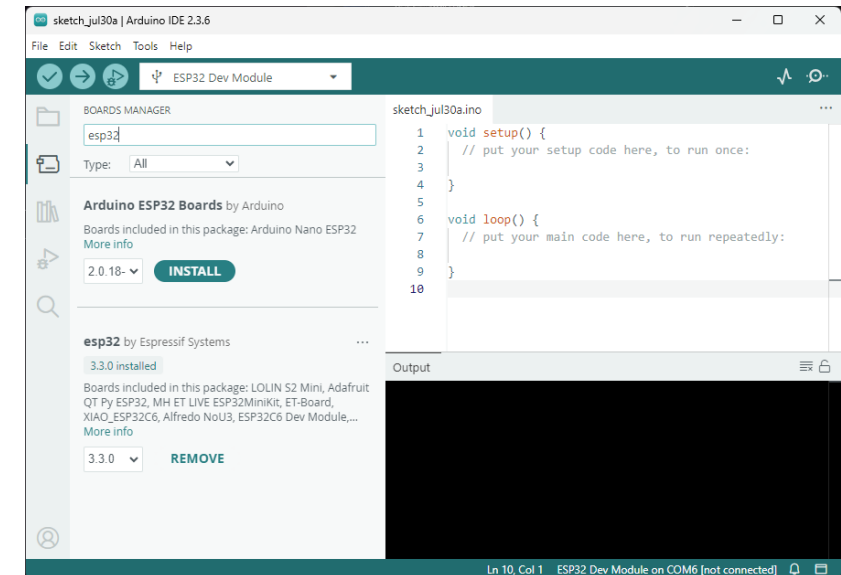
Arduino IDE – Depuración

- La herramienta de depuración se utiliza para probar y depurar programas.
- Se puede utilizar para navegar por la ejecución de un programa de forma controlada.



Arduino IDE – Configurar ESP32

- El ESP32 no viene incluido en el IDE de Arduino por defecto, pero es posible agregarlo utilizando el Gestor de tarjetas:
 - Menú Archivo > Preferencias. En el campo “URLs adicionales de gestor de tarjetas”, añadir la siguiente URL:
 - https://espressif.github.io/arduino-esp32/package_esp32_index.json
 - Menú Herramientas > Placa > Gestor de tarjetas.
 - En el panel que aparece, buscar “ESP32” en la barra de búsqueda, y pulsar en “Instalar” en el paquete que aparece.
- Una vez completada la instalación, ya se podrá elegir los modelos de ESP32 en el selector de placa.



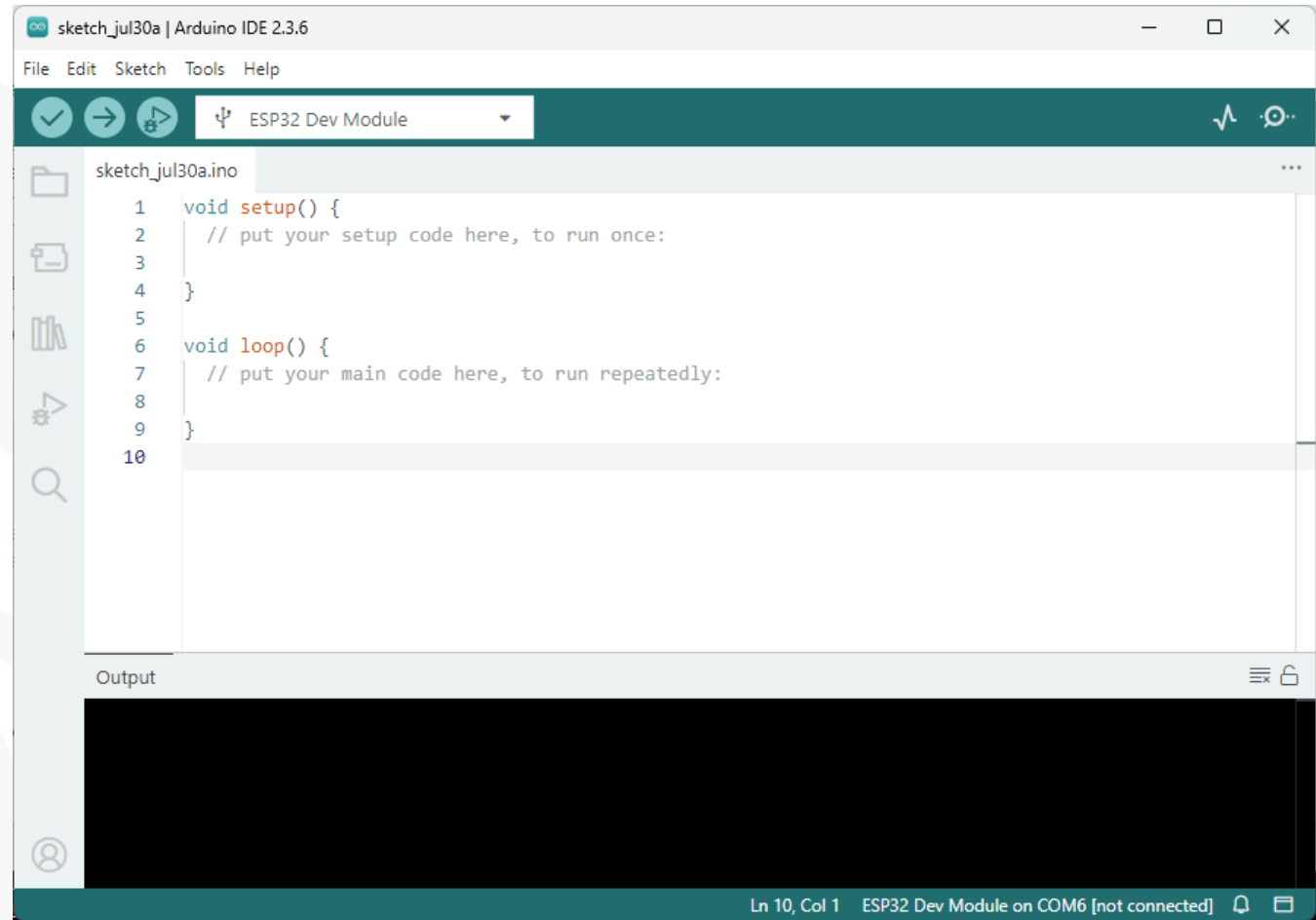
Arduino IDE – Configurar ESP32

- Para verificar el correcto funcionamiento:
 - Conectar el ESP32 mediante un cable USB.
 - En el IDE de Arduino, seleccionar la placa a utilizar.
 - Herramientas > Placa y seleccionar “ESP32 Dev Module”.
 - Seleccionar el puerto COM al que está conectado el ESP32:
 - Herramientas > Puerto
 - Si no se conoce qué puerto es se recomienda desconectar el ESP32 y volver a esta sección para ver qué puerto desaparece y vuelve a aparecer cuando se vuelve a conectar el dispositivo.

Programar en Arduino

Estructura básica de un programa en Arduino

- Los programas de Arduino presentan 2 funciones principales:
 - setup: se ejecuta una vez cuando el Arduino enciende
 - loop: se ejecuta continuamente mientras está encendido



The screenshot displays the Arduino IDE 2.3.6 interface. The main editor window shows the file `sketch_jul30a.ino` with the following code:

```
1 void setup() {  
2   // put your setup code here, to run once:  
3  
4 }  
5  
6 void loop() {  
7   // put your main code here, to run repeatedly:  
8  
9 }  
10
```

The interface includes a menu bar (File, Edit, Sketch, Tools, Help), a toolbar with icons for checking, running, and uploading, and a dropdown menu for the board type set to "ESP32 Dev Module". The status bar at the bottom indicates "Ln 10, Col 1" and "ESP32 Dev Module on COM6 [not connected]".

Estructura básica de un programa

```
// ----- ADJUSTS ----- */
#define PIN_SENSOR 13

/* ----- LIBRARIES ----- */
#include <Servo.h>
#include "infoextra.cpp"

/* ----- VARIABLES/CONST ----- */
int temperature;
const float calibrationValue = 1.48;

/* ----- SETUP ----- */
void setup(){}

/* ----- LOOP ----- */
void loop(){}

/* ----- OTHER FUNCTIONS ----- */
void myFunction(){}

```

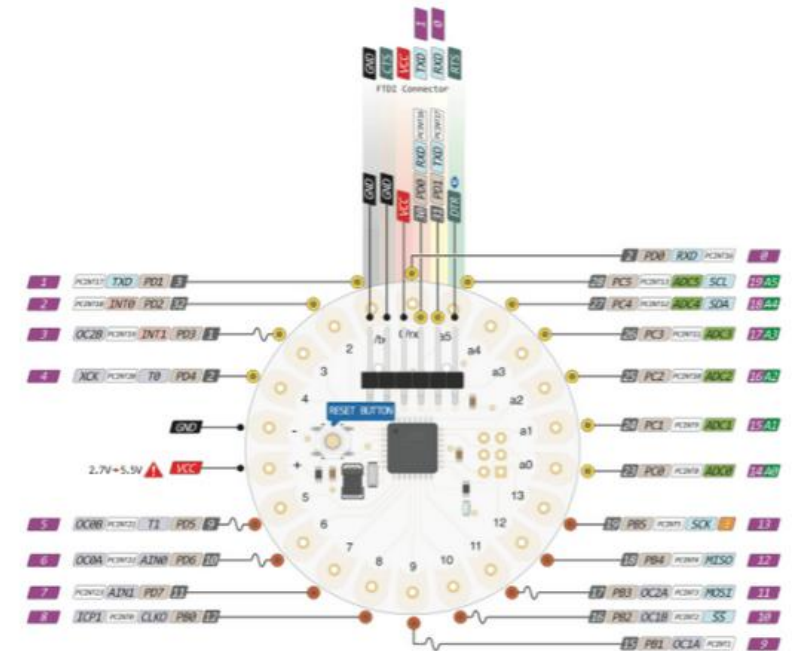
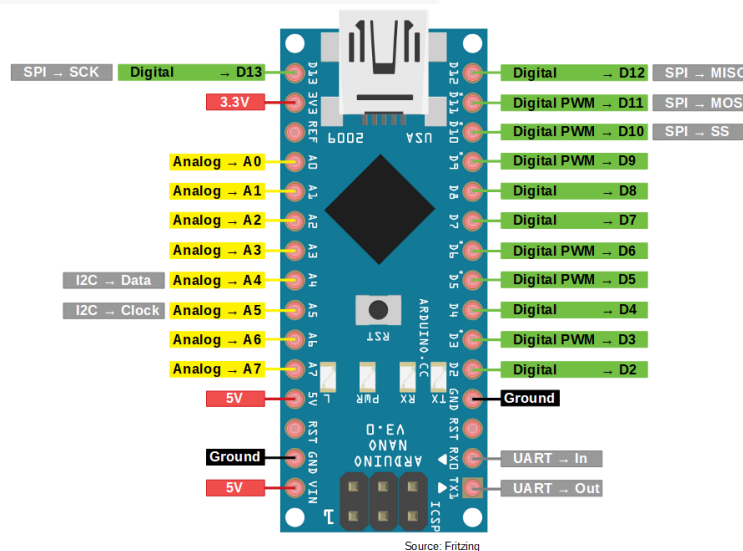
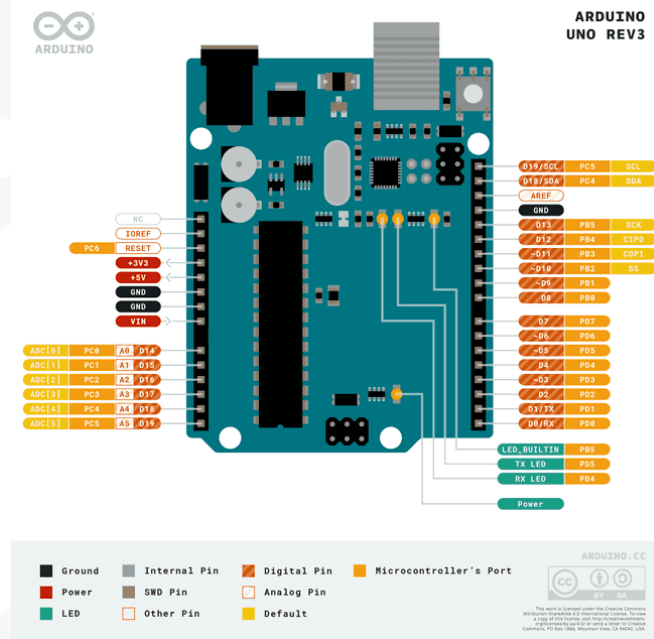
Ejecutar un programa

- Selección de placa y puerto:
 - Abre el IDE de Arduino.
 - Ve a "Herramientas" > "Placa" y selecciona el modelo de tu placa Arduino (por ejemplo, Arduino Uno).
 - Ve a "Herramientas" > "Puerto" y selecciona el puerto serial al que está conectada la placa.
- Carga del programa:
 - Abre el programa (sketch) que deseas ejecutar en el IDE.
 - Haz clic en el botón "Subir" (flecha hacia la derecha) para cargar el programa en la placa.
- Verificación:
 - Una vez que el programa se ha subido con éxito, la placa Arduino comenzará a ejecutarlo.
 - Si tu programa interactúa con la computadora (por ejemplo, a través de la comunicación serial), puedes abrir el Monitor Serial en el IDE para ver los datos.

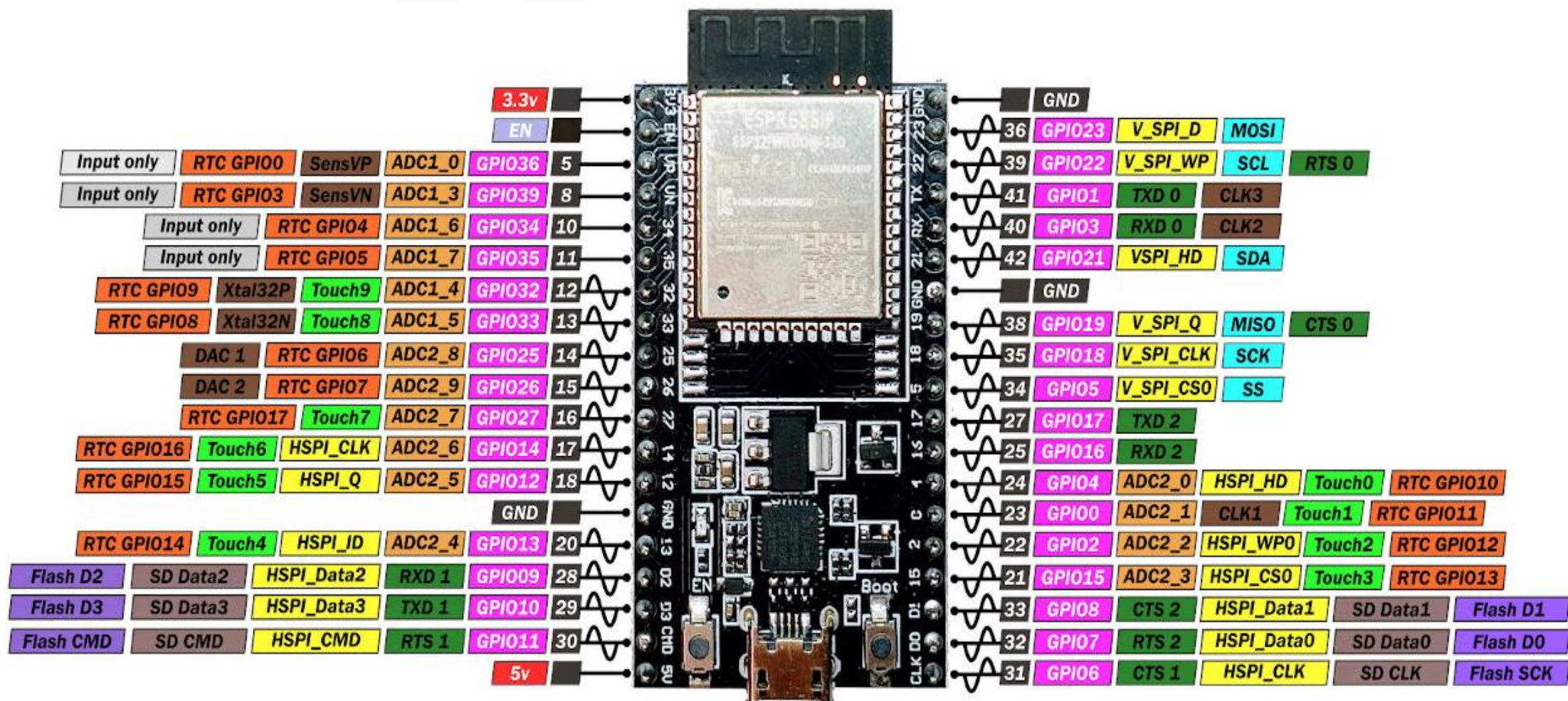
Conexiones

Conexiones placa

- Para conocer la funcionalidad de cada uno de los pines de la placa es necesario contar con el Pinout que será específico de cada placa



ESP32



Pines especiales en el ESP32

- Pines de configuración:
 - Intervienen en la configuración durante el arranque
- Pines GPIO RTC
 - Se utilizan para despertar el ESP32 del modo de bajo consumo
- GPIO de Entrada Únicamente
 - Solo pueden emplearse como entradas digitales o analógicas
- Pines táctiles
 - Cuando una carga capacitiva está cerca del pin detecta el cambio en la capacitancia
- Pin Enable (EN)
 - Controla el regulador de 3V3
- Pines de PWM
 - Pueden utilizarse para controlar motores y Leds digitales

Pines especiales en el ESP32

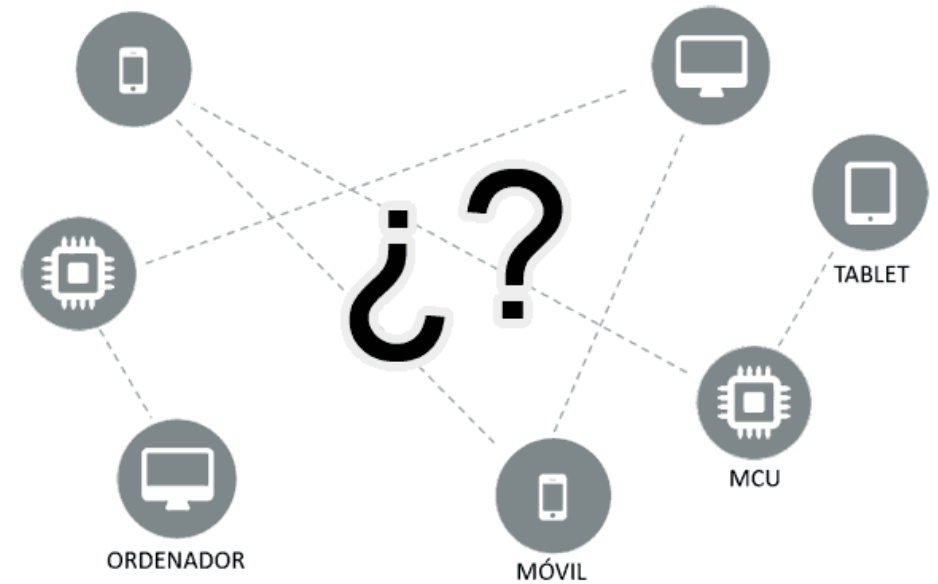
- Pines ADC
 - Sirven para leer señales analógicas y convertirlas en valores digitales
- Pines DAC
 - Permite convertir señales digitales en señales analógicas
- Pines UART
 - Permite establecer comunicación serie asíncrona entre placa y otros dispositivos
- Pines I2C
 - Pines asociados al bus I2C. Por defecto incluye los siguientes pines: SDA y SCL
- Pines SPI
 - Permite la comunicación serial síncrona de alta velocidad con protocolo SPI

Computación Ubicua

Arduino y MQTT

Protocolos de Comunicación

- Un protocolo de comunicación es una serie de normas que definimos para que dos o más dispositivos puedan comunicarse y entenderse.

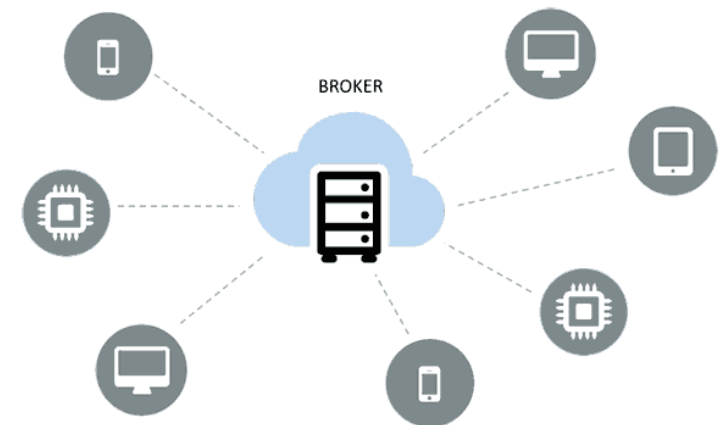


Condicionantes en IoT

- Posibilidad de tener una gran cantidad de dispositivos interconectados
- Escalabilidad del sistema: Que puedan añadirse o retirarse dinámicamente dispositivos sin que el comportamiento global se modifique
- Débil acoplamiento entre dispositivos: la dependencia entre los dispositivos debe ser la menor posible, y deseablemente nula
- Gran número de comunicaciones simultáneas: Respuestas rápidas gracias a mensajes pequeños y que no requieran gran procesamiento
- Seguridad: Dispositivos expuestos a Internet y que transmiten información privada e incluso controlan sistemas físicos

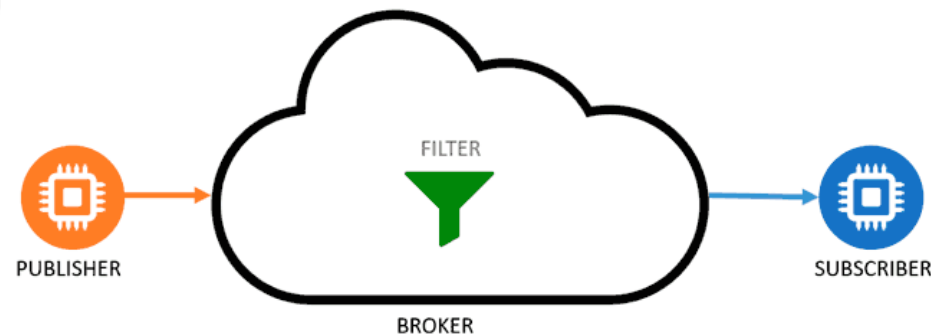
Soluciones de comunicación en IoT

- Externalizar la comunicación un servicio de notificaciones centralizado
- La solución consiste en disponer **un servidor central que se encarga de recibir los mensajes de todos los dispositivos emisores, y distribuirlos a los receptores.** De forma genérica llamaremos a este servidor 'Router' o 'Broker'.



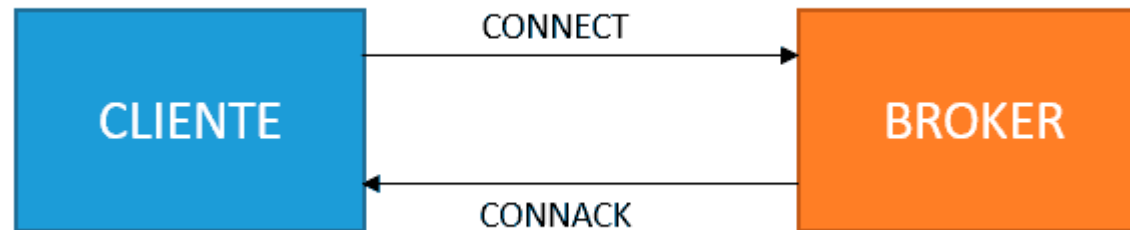
MQTT

- MQTT (Message Queing Telemetry Transport). Es un protocolo de comunicación M2M (machine-to-machine) de tipo message queue (Cola de mensajes).
- El funcionamiento del MQTT es un servicio de mensajería push con patrón publicador/suscriptor (pub-sub).
- Para filtrar los mensajes que son enviados a cada cliente los mensajes se disponen en topics organizados jerárquicamente.



MQTT

- Los clientes inician una conexión TCP/IP con el bróker, el cual mantiene un registro de los clientes conectados. Esta conexión se mantiene abierta hasta que el cliente la finaliza.
- Para conectar el cliente envía un mensaje CONNECT que contiene información necesaria (nombre de usuario, contraseña, client-id...). El broker responde con un mensaje CONNACK, que contiene el resultado de la conexión (aceptada, rechazada, etc).

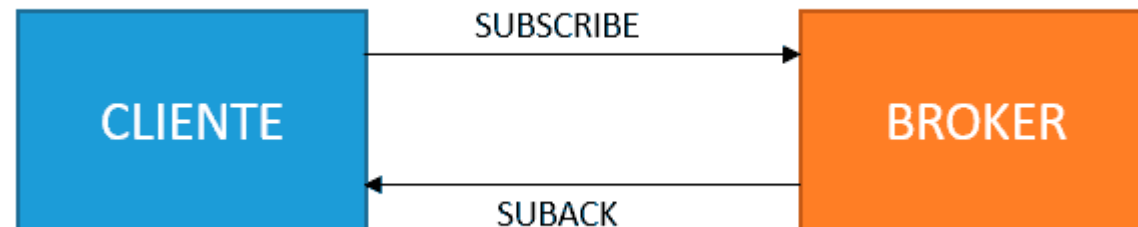


MQTT

- Para enviar los mensajes el cliente emplea mensajes PUBLISH, que contienen el topic y el payload.



- Para suscribirse y desuscribirse se emplean mensajes SUBSCRIBE y UNSUBSCRIBE, que el servidor responde con SUBACK y UNSUBACK.



MQTT

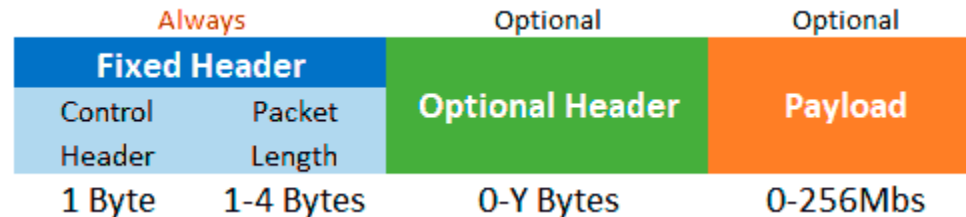
- Para asegurar que la conexión está activa los clientes mandan periódicamente un mensaje PINGREQ que es respondido por el servidor con un PINGRESP. Finalmente, el cliente se desconecta enviando un mensaje de DISCONNECT.

Calidad del servicio (QoS)

- MQTT tiene tres niveles QoS posibles:
 - **QoS 0 unacknowledged (at most one)**: El mensaje se envía una única vez. En caso de fallo por lo que puede que alguno no se entregue.
 - **QoS 1 acknowledged (at least one)**: El mensaje se envía hasta que se garantiza la entrega. En caso de fallo, el suscriptor puede recibir algún mensaje duplicado.
 - **QoS 2 assured (exactly one)**. Se garantiza que cada mensaje se entrega al suscriptor, y únicamente una vez.
- Tanto en QoS 1 como en QoS 2, los mensajes se guardan o se ponen en cola para los clientes que están desconectados y que tienen una sesión persistente establecida. Estos mensajes se reenvían una vez que el cliente vuelve a conectarse
- Usar un nivel u otro depende de las características y necesidades de fiabilidad de nuestro sistema.

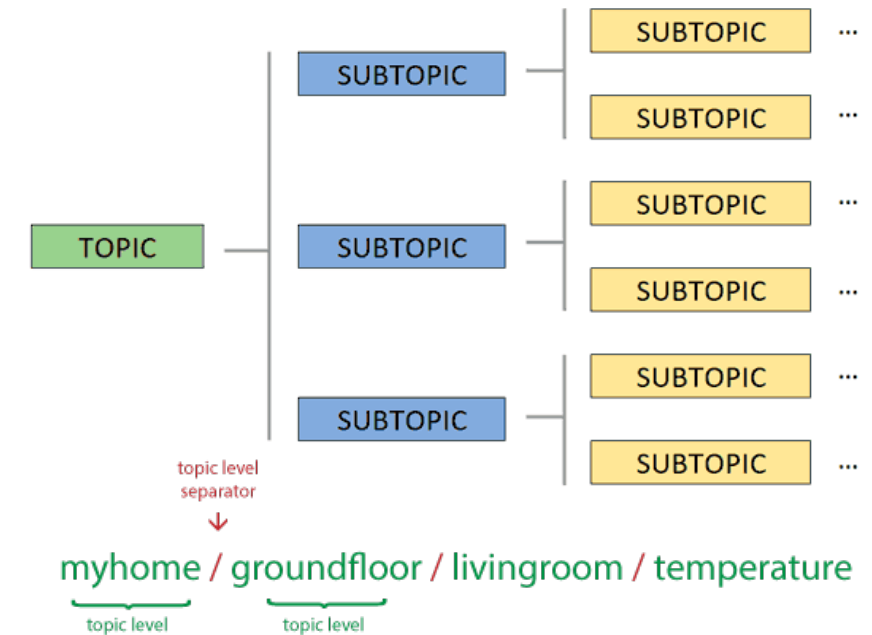
MQTT – Estructura de mensajes

- Cada mensaje consta de 3 partes:
 - **Cabecera fija.** Ocupa de 2 a 5 bytes, obligatorio. Consta de:
 - Código de control, que identifica el tipo de mensaje enviado.
 - Longitud del mensaje, que se codifica en 1 a 4 bytes, de los cuales se emplean los 7 primeros bits, y el último es un bit de continuidad.
 - **Cabecera variable.** Opcional, contiene información adicional que es necesaria en ciertos mensajes o situaciones.
 - **Contenido(payload).** contenido real del mensaje. Puede tener un máximo de 256 Mb aunque en implementaciones reales el máximo es de 2 a 4 kB.



Topics en MQTT

- **Los topics son estructuras** que utilizan el carácter de barra (/) como delimitador similar a la de un árbol de directorios.
- **El Broker acepta todos los Topics.** No es necesario crearlo explícitamente antes de publicar o suscribirse al Broker.
- **Los clientes pueden suscribirse a uno o varios Topic.** Para ello, el cliente puede establecer varias suscripciones y/o emplear Wildcards
- **Los clientes publican mensajes indicando un único Topic.** El Broker recibe el mensaje y, si encuentra alguna suscripción que cumpla con el filtro del Topic, transmite el mensaje a los clientes suscritos.



MQTT y Arduino

- Es necesario disponer de un Broker previamente configurado
- Instalar la librería PubSubClient en el IDE de Arduino
- Árbol de Topics a utilizar
- Se ha estructurado en 5 partes:
 - main.ino: sketch principal que inicia WiFi y la comunicación MQTT
 - config.h: configuración de la conexión WiFi
 - ESP32_Utils.hpp: contiene funciones que genéricas del ESP32
 - ESP32_Utils_MQTT.hpp: contiene funciones que genéricas para comunicación en MQTT
 - MQTT.hpp: funciones que sí son específicas del proyecto y definiciones del Broker

Ejemplo (main.ino)

```
#include <WiFi.h>
#include <AsyncMqttClient.h>
#include <ArduinoJson.h>

#include "config.h" // Sustituir con datos de
                    // vuestra red
#include "MQTT.hpp"
#include "ESP32_Utils.hpp"
#include "ESP32_Utils_MQTT_Async.hpp"

void setup(void)
{
    Serial.begin(115200);
    delay(500);
    WiFi.onEvent(WiFiEvent);
    InitMqtt();
    ConnectWiFi_STA();
}

void loop()
{
    PublishMqtt();
    delay(500);
}
```


Ejemplo (config.h)

```
const char* ssid      = "ssid";  
const char* password = "password";  
const char* hostname = "ESP32_1";  
  
IPAddress ip(192, 168, 1, 200);  
IPAddress gateway(192, 168, 1, 1);  
IPAddress subnet(255, 255, 255, 0);
```

Ejemplo (ESP32_Utils.hpp)

```
void ConnectWiFi_STA(bool useStaticIP = false)
{
    Serial.println("");
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);
    if(useStaticIP) WiFi.config(ip, gateway, subnet);
    while (WiFi.status() != WL_CONNECTED)
    {
        delay(100);
        Serial.print('.');
    }

    Serial.println("");
    Serial.print("Iniciado STA:\t");
    Serial.println(ssid);
    Serial.print("IP address:\t");
    Serial.println(WiFi.localIP());
}
```

```
void ConnectWiFi_AP(bool useStaticIP = false)
{
    Serial.println("");
    WiFi.mode(WIFI_AP);
    while(!WiFi.softAP(ssid, password))
    {
        Serial.println(".");
        delay(100);
    }
    if(useStaticIP) WiFi.softAPConfig(ip, gateway, subnet);

    Serial.println("");
    Serial.print("Iniciado AP:\t");
    Serial.println(ssid);
    Serial.print("IP address:\t");
    Serial.println(WiFi.softAPIP());
}
```

Ejemplo (ESP32_Utils_MQTT.hpp)

```
void InitMqtt()
{
    mqttClient.setServer(MQTT_BROKER_ADDRESS, MQTT_PORT);
    Suscribirse();
    mqttClient.setCallback(OnMqttReceived);
}

void ConnectMqtt()
{
    while (!mqttClient.connected())
    {
        Serial.print("Starting MQTT connection...");
        if (mqttClient.connect(MQTT_CLIENT_NAME))
        {
            Suscribirse();
        }
        else
        {
            Serial.print("Failed MQTT connection, rc=");

            Serial.print(mqttClient.state());
            Serial.println(" try again in 5 seconds");

            delay(5000);
        }
    }
}

void HandleMqtt()
{
    if (!mqttClient.connected())
    {
        ConnectMqtt();
    }
    mqttClient.loop();
}
```

Ejemplo (MQTT.hpp)

```
const char* MQTT_BROKER_ADDRESS = "192.168.1.150";
const uint16_t MQTT_PORT = 1883;
const char* MQTT_CLIENT_NAME = "ESP8266Client_1";

WiFiClient espClient;
PubSubClient mqttClient(espClient);

void SuscribeMqtt()
{
    mqttClient.subscribe("hello/world");
}

String payload;
void Publismqtt(unsigned long data)
{
    payload = "";
    payload = String(data);
    mqttClient.publish("hello/world", (char*)payload.c_str());
}
```

```
String content = "";
void OnMqttReceived(char* topic, byte* payload, unsigned int
length)
{
    Serial.print("Received on ");
    Serial.print(topic);
    Serial.print(": ");

    content = "";
    for (size_t i = 0; i < length; i++) {
        content.concat((char)payload[i]);
    }
    Serial.print(content);
    Serial.println();
}
```

Cada tipo de sensor genera un JSON con:

- **sensor_id**: Identificador único del sensor.
- **sensor_type**: Tipo de sensor (weather, traffic_counter, traffic_light, information_display).
- **street_id**: Calle asociada al sensor.
- **timestamp**: Marca de tiempo del dato generado.
- **location**: Información de ubicación (latitud, longitud, distrito, barrio, etc.).
- **data**: Datos específicos del sensor, que varían según el tipo.

Weather_stations

```
{  
  "sensor_id": "WS_001",  
  "sensor_type": "weather",  
  "street_id": "ST_001",  
  "timestamp": "2025-10-15T12:34:56.789Z",  
  "location": {  
    "latitude": 40.416775,  
    "longitude": -3.703790,  
    "altitude_meters": 650.0,  
    "district": "Centro",  
    "neighborhood": "Sol"  
  },  
  "data": {  
    "temperature_celsius": 22.5,  
    "humidity_percent": 65.0,  
    "air_quality_index": 45,  

```

```
    "wind_speed_kmh": 12.3,  
    "wind_direction_degrees": 180,  
    "atmospheric_pressure_hpa": 1015.2,  
    "uv_index": 5  
  }  
}
```

traffic_counter

```
{  
  "sensor_id": "TC_001",  
  "sensor_type": "traffic_counter",  
  "street_id": "ST_002",  
  "timestamp": "2025-10-15T12:34:56.789Z",  
  "location": {  
    "latitude": 40.416775,  
    "longitude": -3.703790,  
    "district": "Centro",  
    "neighborhood": "Sol"  
  },  
  "data": {  
    "vehicle_count": 35,  
    "pedestrian_count": 120,  
    "bicycle_count": 5,  
    "direction": "north",  
    "counter_type": "vehicle",  
    "technology": "camera",  
    "average_speed_kmh": 28.5,  
  }  
}
```

```
    "occupancy_percentage": 75.0,  
    "traffic_density": "high"  
  }  
}
```

traffic_light

```
{  
  "sensor_id": "TL_001",  
  "sensor_type": "traffic_light",  
  "street_id": "ST_003",  
  "timestamp": "2025-10-15T12:34:56.789Z",  
  "location": {  
    "latitude": 40.416775,  
    "longitude": -3.703790,  
    "district": "Centro",  
    "neighborhood": "Sol"  
  },  
  "data": {  
    "current_state": "green",  
    "cycle_position_seconds": 15,  
    "time_remaining_seconds": 45,  
    "cycle_duration_seconds": 90,  
    "traffic_light_type": "mixed_vehicle_pedestrian",  
    "circulation_direction": "bidirectional",  
    "pedestrian_waiting": false,  
    "pedestrian_button_pressed": false,  
    "malfunction_detected": false,  
  }  
}
```

```
    "cycle_count": 3,  
    "state_changed": true,  
    "last_state_change": "2025-10-15T12:34:11.789Z"  
  }  
}
```

information_display

```
{  
  "sensor_id": "ID_001",  
  "sensor_type": "information_display",  
  "street_id": "ST_004",  
  "timestamp": "2025-10-15T12:34:56.789Z",  
  "location": {  
    "latitude": 40.416775,  
    "longitude": -3.703790,  
    "district": "Centro",  
    "neighborhood": "Sol"  
  },  
  "data": {  
    "display_status": "active",  
    "current_message": "Tráfico fluido",  
    "content_type": "traffic",  
    "brightness_level": 85,  
    "display_type": "led_matrix",  
    "display_size_inches": 420.0,  
    "supports_color": true,  
    "temperature_celsius": 45.3,  
  }  
}
```


"energy_consumption_watts": 250.5,

"last_content_update": "2025-10-15T12:34:56.789Z"

}

}